CodeFusionAI — Infosys Interview Complete Guide

Simple English Explanations • Frontend Live-Coding • CS Fundamentals • System Design

1. The 5 Points on Your Resume (Read & Remember)

- **1. Real-Time Collaboration:** Built a real-time collaborative code editor using Socket.IO and Monaco Editor, allowing multiple developers to edit code together live with under 50ms delay.
- **2. In-Browser Execution (MERN):** Implemented an in-browser runtime using WebContainers, allowing users to install npm packages and run full-stack MERN projects directly in the browser with a live preview.
- **3. Redis Caching & Cloud Savings:** Reduced AWS S3 cloud costs by 85% by caching live edits in Redis and saving files in batches every 30 seconds instead of on every keystroke.
- **4. Gemini AI Integration:** Integrated Google Gemini AI to automatically review code, detect bugs, explain time and space complexity, and answer coding questions.
- **5. Auth & Offline Save:** Implemented Firebase Auth with Google Sign-In for secure login and used IndexedDB to auto-save work locally so users never lose their code.

2. How to Explain the Project in 30 Seconds (Simple English)

Say this when they say: 'Tell me about yourself / your project'

"Sir, CodeFusionAI is an online code editor like Google Docs or Replit, built for MERN stack projects. Multiple developers can join a room and write code together live without lag using WebSockets. Instead of running servers on our backend, users can run React and Node.js directly inside their own browser using WebContainers. We also connected Google Gemini AI for instant code review, and we used Redis to cache edits and batch saves to AWS S3, which reduced cloud storage costs by 85%."

3. Frontend Deep Dive & Live Coding Questions (React & JS)

In Infosys, the interviewer will often ask you to **open Notepad, share your screen, or write code on paper**. Here are the exact questions they ask for Frontend:

Q1: 'Write a Debounce function in JavaScript.' (Very Common!)

Why they ask this: When a user types in Monaco Editor, typing 50 letters in 10 seconds shouldn't send 50 socket events. We debounce it so it sends only after the user pauses.

```
// Simple Debounce function
function debounce(fn, delay = 300) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

// How to use in React:
const sendCodeChange = debounce((newCode) => {
  socket.emit("code-change", { roomId, code: newCode });
}, 300);
```

Q2: 'Write a React component that connects to a WebSocket and avoids memory leaks.'

Why they ask this: To check if you know how `useEffect` cleanup works. If you don't remove socket listeners, you get duplicate events and memory leaks!

```
import { useEffect, useState } from "react";
import { socket } from "../services/socket";

function CodeRoom({ roomId }) {
  const [code, setCode] = useState("");

  useEffect(() => {
    // 1. Join the room
    socket.emit("join-room", { roomId });

    // 2. Listen for code updates from peers
    const handleReceiveCode = (newCode) => {
      setCode(newCode);
    };
    socket.on("receive-code", handleReceiveCode);

    // 3. CLEANUP FUNCTION (Crucial! Prevents memory leaks)
    return () => {
      socket.off("receive-code", handleReceiveCode);
    };
  }, [roomId]);

  return <div>{/* Editor rendering */}</div>;
}
```

Q3: 'How did you integrate Monaco Editor inside React?'

Answer in simple words: *'We used `@monaco-editor/react`. We pass properties like `language`, `theme`, and `value`. When the user types, the `onChange` event fires. We also get the editor instance in `onMount` so we can control cursor positions and selections.'*

```
import Editor from "@monaco-editor/react";

<Editor
  height="90vh"
  language="javascript"
  theme="vs-dark"
  value={code}
  onChange={(newValue) => {
    setCode(newValue);
    socket.emit("code-change", { roomId, code: newValue });
  }}
  onMount={(editor, monaco) => {
    console.log("Monaco Editor is ready!");
  }}
/>
```

Q4: 'What is the difference between useState, useEffect, useRef, and useCallback?'

- **useState:** Stores data that re-renders the UI when changed (e.g., current code text, active file name).
- **useEffect:** Runs side effects (e.g., connecting to WebSockets, loading files from IndexedDB when the page loads).
- **useRef:** Stores a reference that stays the same WITHOUT re-rendering the UI (e.g., holding the editor instance, terminal DOM node, or drag coordinates).
- **useCallback:** Caches a function definition between re-renders so child components don't re-render unnecessarily.

Q5: 'How did you make the terminal and sidebar resizable in React?'

Answer: We store `sidebarWidth` in state. When the user clicks the divider, we listen to mouse movements (`mousemove`) and update the width. On mouse release (`mouseup`), we stop listening.

4. Core CS Fundamentals (Simple English Answers)

Infosys checks your basics. Speak these simple 2-sentence answers confidently:

1. Networking: Why WebSockets instead of HTTP?

Simple Answer: HTTP sends a new request and closes every time, which has high delay (lag) and large headers. WebSockets open one continuous connection like a phone call, so messages travel in under 50ms.

2. Networking: Does Socket.IO use TCP or UDP?

Simple Answer: It uses **TCP** because code editing cannot lose any letters or semicolons. TCP guarantees that every single packet arrives in the exact correct order.

3. OS: Node.js is single-threaded. How does it handle multiple users?

Simple Answer: Node.js runs your JavaScript code on one thread, but it gives heavy tasks (like network sockets, database queries, and timers) to the background OS worker threads using the **Event Loop**. That is why it never freezes.

4. OS: What is a Race Condition, and how did you solve it?

Simple Answer: A race condition is when two users save at the exact same millisecond and one person's changes accidentally overwrite the other's. We solved it by saving each file under its own key in a **Redis Hash**, and using **Redis Mutex Locks** so only one save runs at a time.

5. DBMS: Why MongoDB instead of SQL for this project?

Simple Answer: Code projects have folders inside folders and files of different sizes. This fits naturally into MongoDB's JSON-like document format. In SQL, nested folders would need slow, complicated JOIN queries.

6. DBMS: What is the difference between Redis and MongoDB?

Simple Answer: Redis stores data in **RAM memory** (super fast, but temporary). MongoDB stores data on **hard disk** (permanent, but slower). We use Redis for live keystrokes, and MongoDB for permanent records.

7. OOPs: Name 2 Design Patterns used in your project.

Simple Answer: Observer Pattern: When code changes, the server broadcasts an event to all users listening in the room. **Singleton Pattern:** Only one database and Redis connection instance is created and shared across the entire server.

5. System Design Questions (Token Bucket & Execution)

How did you measure the sub-50ms latency?

Simple Answer: I added a timestamp when the person typing pressed a key. When the other user received it on their screen, I took a second timestamp. The time difference was consistently between 35ms and 50ms.

Explain the Token Bucket rate limiter in simple words.

Simple Answer: It is like a bucket with 5 tickets. Every 5 seconds, 1 new ticket is added. When a user types 10 letters, we check if a ticket is available. If yes, we save the file to AWS S3 and take 1 ticket. If no, we wait. This stopped us from making 1,000s of expensive S3 calls.

Does it save automatically or manually?

Simple Answer: Both! It automatically saves in the background after 10 edits and every 30 seconds. But the user can also press Ctrl+S to save manually anytime.

What is WebContainers?

Simple Answer: It is a technology that runs Node.js and React directly inside the user's web browser using WebAssembly. We don't have to pay for backend servers to run user code, and it starts up instantly.

What is IndexedDB?

Simple Answer: It is a database inside your web browser with lots of storage. We use it to auto-save files locally so if the user refreshes the page or loses Wi-Fi, their code is still there.

6. Quick Live-Coding Checklist (If they ask you to write code)

If the interviewer says: *'Open a plain editor and write this'*, here are 3 quick snippets to memorize:

A. JavaScript Array Methods (map, filter, reduce)

```
// 1. map (transform items)
const fileNames = files.map(f => f.path);

// 2. filter (select items)
const codeFiles = files.filter(f => !f.isFolder);

// 3. reduce (calculate total)
const totalSize = files.reduce((acc, f) => acc + (f.size || 0), 0);
```

B. Fetching AI Review from Backend (async/await + try/catch)

```
async function getAICodeReview(codeSnippet) {
  try {
    const response = await fetch("/api/ai/review", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ code: codeSnippet, language: "javascript" }),
    });

    if (!response.ok) throw new Error("Server error");
    const data = await response.json();
    return data.review; // Markdown response from Gemini
  } catch (error) {
    console.error("AI Review failed:", error.message);
    return "Error getting review. Please try again.";
  }
}
```

7. Infosys Golden Interview Rules

Remember for Tomorrow

1. **Never rush your answer:** Take 3 seconds to think before speaking.
2. **Say 'I built this' with confidence:** Speak about Redis, WebSockets, and React as decisions you made.
3. **If you don't know a syntax:** Say: *'I know the logic: we need to use a debounce timer / useEffect cleanup, but I might need to verify the exact method name.'* Interviewers respect honesty and problem-solving!